

Kubernetes Complete Notes

By Mr. Riaz Sir | Prepared for Interview + Practical Use

ENVIRONMENT VARIABLES (ENV)

One Line to Remember: ENV variables are the way we tell our application its settings (like database passwords or usernames) without hardcoding them into the code.

The Restaurant Menu Analogy 📜

Imagine a kitchen (Pod) where the Chef (Container) needs to know the "Price of the Day" or the "Special Ingredient." Instead of writing it on the wall of the kitchen permanently, the Manager (Kubernetes) writes it on a whiteboard (Environment Variable). If the Price changes, the Manager just updates the whiteboard.

Sir's Notes — Environment Variables

ENV VARIABLES: It is a way to pass configuration information to containers running within pods. To set Env vars it include the `env` or `envFrom` field in the configuration file.

- ENV: Allows you to set environment variables for a container, specifying a value directly for each variable through CLI / Command prompt.
 - ENVFROM: Passing variables from a file. There are 2 types: ConfigMaps and Secrets.
-

CONFIGMAPS (CM)

One Line to Remember: ConfigMaps are for public information (Port numbers, App names). They are NOT secure.

Sir's Notes — ConfigMaps

It is used to store data in key-value pairs, files, or command-line arguments that can be used by pods and containers in a cluster.

- Rule: The data should be non-confidential. It does not provide security or encryption.
- Limit: The limit of a ConfigMap is only 1MB.
- Note: If you want to store more than 1MB, use a mount volume or a separate database/file service.

My Explanation — The 1MB Rule: Think of a ConfigMap like a post-it note. You can stick a few settings on it, but you can't write a whole book on it. For "books" (large data), use Volumes.

Sir's Notes — Practical Lab (ConfigMaps)

1. Creating a Deployment that will crash (MariaDB needs a password):

```
kubectl create deploy newdb --image=mariadb
kubectl get pods
# Status: CrashLoopBackOff (It crashed because no password was provided)
```

2. Fixing it by passing ENV directly (The hard way):

```
kubectl set env deploy newdb MYSQL_ROOT_PASSWORD=root123456
kubectl get pods
# Status: Running ✓
```

3. Fixing it using ConfigMaps (The professional way): Step A: Create a file called vars

```
vi vars
# Inside the file:
MYSQL_ROOT_PASSWORD=root123456
MYSQL_USER=admin
```

Step B: Create the ConfigMap from that file

```
kubectl create cm dbvars --from-env-file=vars
kubectl get cm
kubectl describe cm dbvars
# This will show the plain data
```

Step C: Attach ConfigMap to Deployment

```
kubectl set env deploy newdb --from=configmaps/dbvars
kubectl get pods
# Status: Running ✓
```

KUBERNETES SECRETS

One Line to Remember: Secrets = ConfigMaps + Encryption (Base64). Use these for passwords, keys, and tokens.

The Digital Vault Analogy

ConfigMap is like writing your phone number on a piece of paper. Anyone can read it. Secret is like writing your bank PIN in a code that only the bank understands. It looks like gibberish (base64) to a normal person.

Sir's Notes — Secrets

SECRETS: To store sensitive data in an unencrypted format like passwords, ssh-keys, etc. It uses base64 encoded format.

WHY: If I don't want to expose sensitive info, we use SECRETS. Note: By default, K8s creates some system secrets used for internal communication. Do not delete them.

Types of Secrets:

1. Generic: Creates secrets from files, directories, or literal values.
 2. TLS: Used for SSL/TLS certificates (HTTPS).
 3. Docker Registry: Used to pull images from private repositories using passwords.
-

Sir's Notes — Practical Lab (Secrets)

1. Create Secret from CLI (Literal):

```
# Literal means direct value
kubectl create secret generic password
--from-literal=ROOT_PASSWORD=reyaz123
```

2. Create Secret from File:

```
kubectl create secret generic my-secret --from-env-file=vars
```

3. Injecting Secret into Deployment:

```
# This will fail initially because MariaDB expects specific
prefixes
kubectl set env deploy newdb --from=secret/my-secret
--prefix=MYSQL_
```

Note: Without the `--prefix=MYSQL_`, the container won't recognize the variable, and the pod will crash.

4. How to Read/Decode a Secret: Secrets are encoded, not encrypted. You can decode them easily:

```
# Step 1: Get the YAML
kubectl get secrets password -o yaml

# Step 2: Copy the encoded string and decode it
echo -n "REtaYno0NTY=" | base64 -d
# Output: reyaz123
```

TROUBLESHOOTING POD STATES

One Line to Remember: If your Pod isn't "Running," it's trying to tell you what's wrong.

State	What it means	How to fix
ImagePullBackOff	K8s cannot find the image or doesn't have permission.	Check image name spelling or add <code>imagePullSecret</code> .
CrashLoopBackOff	Container started but immediately died (usually missing ENV).	Check <code>kubectl logs <pod-name></code> .

Sir's Notes — State Explanations

- **ImagePullBackOff:** When a kubelet starts creating containers, it might stay in a waiting state. Reasons: Invalid image name or pulling from a private registry without a secret. K8s will keep trying with an increasing delay (up to 5 mins).
- **CrashLoopBackOff:** Kubelet tries to run the container, but it keeps crashing. K8s enters a loop of restarts.

Interview Questions — ConfigMaps & Secrets

Q: Difference between ConfigMap and Secret? ConfigMaps are for plain-text configuration (non-sensitive). Secrets are for sensitive data (passwords) and are stored in `base64` encoded format.

Q: What is the size limit for a ConfigMap? 1 Megabyte (1MB).

Q: How do you see the actual value of a Secret? You use `kubectl get secret <name> -o yaml` to get the base64 string, then use the `base64 --decode` command in Linux to see the plain text.

Q: Why would a MariaDB pod show `CrashLoopBackOff`? Commonly because it requires an environment variable (like `MYSQL_ROOT_PASSWORD`) to start. If you don't provide it via `env` or `envFrom`, the database process exits immediately.

Q: What is the benefit of using `--from-env-file`? It allows you to manage hundreds of variables in a single local file and upload them all at once into Kubernetes instead of typing them one by one in the terminal.

POD DESIGN PATTERNS (Sidecar, Adapter, Ambassador, Init Container)

SIDE CAR PATTERN

One Line to Remember:

Sidecar = Helper container that runs with the main container inside the same Pod.

The Bike With a Sidecar Analogy 🏍️

Imagine a bike (Main Container).

Now attach a sidecar to it. The bike runs the main engine. The sidecar does supporting work like:

- Carry luggage
- Carry another passenger
- Support main rider

They move together. They stop together. They share the same fuel (network + volume).

That is Sidecar Pattern.

It creates a helper container to the main container.

Main container:

- Runs the application

Helper container:

- Helps the main container

A sidecar container is a secondary container that runs alongside the main application container within the same Kubernetes Pod.

This design pattern allows you to extend or enhance the functionality of the main application without modifying its code.

Common Use Cases:

- Log aggregation
 - Data synchronization
 - Proxying network traffic
 - Monitoring agents
-

Adapter Design Pattern

One Line to Remember:

Adapter = Standardizes the output of the main container.

Sir's Notes:

Adapter design pattern is used to standardize the output pattern of the main container.

Example: If main app logs are in custom format, Adapter converts logs into standard JSON format for logging system.

Ambassador Design Pattern

One Line to Remember:

Ambassador = Connects container to the outside world.

Sir's Notes:

Ambassador pattern is used to connect containers with the outside world.

Example:

- Proxy to external database
 - API gateway sidecar
 - Service mesh proxy (like Envoy)
-

Init Container

One Line to Remember:

Init Container runs first, finishes its work, then exits.

Sir's Notes:

Init container initializes the first work and exits later.

Important:

- Runs before main container
- Must complete successfully
- If fails → Pod will not start

Example:

- Download config files
 - Run DB migrations
 - Check if DB is ready
-

Practical Example — Sidecar Log Aggregation

One Line to Remember:

Main container writes logs → Sidecar serves logs using Nginx.

Scenario:

Main application writes logs to a shared volume. Sidecar container serves those logs over HTTP using Nginx.

YAML File

Create file:

```
vi sidecar.yml
```

sidecar.yml

```
apiVersion: v1
kind: Pod
metadata:
  name: log-aggregator-pod
spec:
  containers:
    - name: app-container
      image: alpine
      command: ["/bin/sh", "-c"]
      args: ["while true; do date >> /var/log/app.log; sleep 5; done"]
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log

    - name: sidecar-container
      image: nginx
      volumeMounts:
        - name: shared-logs
          mountPath: /usr/share/nginx/html

  volumes:
    - name: shared-logs
      emptyDir: {}
```

How It Works — Step by Step

Step 1: Pod is Created


```
kubectl apply -f sidecar.yml
```

Step 2: Two Containers Start

- app-container (Alpine)
- sidecar-container (Nginx)

Step 3: Shared Volume Created

```
emptyDir: {}
```

This creates a temporary shared directory.

What is emptyDir?

One Line to Remember:

emptyDir = Temporary shared storage for containers in same Pod.

- Created when Pod starts
 - Deleted when Pod is deleted
 - Shared between containers inside same Pod
-

Flow Explanation

app-container:

- Runs infinite loop
- Writes current date every 5 seconds
- File: `/var/log/app.log`

sidecar-container:

- Mounts same volume
- Path: `/usr/share/nginx/html`
- Nginx serves that directory over HTTP

Result: Open browser → You can see live logs 

Visual Flow

Main Container

- writes logs to shared volume
- Sidecar reads same volume
- Nginx serves logs via HTTP

Both containers:

- Share network
 - Share volume
 - Run inside same Pod
 - Same lifecycle
-

Sidecar vs Init Container

Feature	Sidecar	Init Container
Runs when	Along with main container	Before main container
Lifecycle	Runs continuously	Exits after completion
Purpose	Support main container	Setup / Initialization
Failure impact	Pod fails if sidecar fails	Pod never starts

Interview Questions

Q1: What is a Sidecar container?

A sidecar container is a helper container that runs alongside the main container in the same Pod. It enhances or extends the functionality of the main container without modifying its code.

Q2: What is the difference between Sidecar and Init container?

Sidecar runs continuously with the main container.
Init container runs first, completes its task, and exits before the main container starts.

Q3: What is emptyDir volume?

emptyDir is a temporary volume created when a Pod starts and deleted when the Pod is removed. It allows containers inside the same Pod to share files.

Q4: What are Adapter and Ambassador patterns?

- Adapter: Standardizes output of main container.
- Ambassador: Connects container to external services using proxy mechanism.